Fusion

— **macOS:** ~/Library/Application Support/Blackmagic Design/Fusion/Fuses

— **Windows:**C:\ProgramData\Blackmagic Design\Fusion\Fuses

— **Linux:** ~/.fusion/BlackmagicDesign/Fusion/Fuses

Restart Resolve and Fusion. This will need to be done to add a new Fuse to the list. Once added, the process of editing, developing can be done on the fly without restarting the applications.

Overview Guide

This Guide section is used with the series of Template Example Fuse Plugins supplied, these have comments inline to explain functionality. These show the functional building blocks and setups to creating custom Fuse plugins. This guide will explain each template Fuse, and the Reference section has further detail of each function.

Starting with background information primer, you can follow along with the Fuse plugins loaded in the Fusion page, loaded in a Text editor, and this.

About the Lua Language

Lua is a powerful, efficient, lightweight, embedded scripting language built into Resolve and Fusion it is used for Scripting and hosting Python scripting as well as Fuse image processing plugins.
It supports procedural programming, object-oriented programming, functional programming, data-driven programming, and data description.

Lua combines simple procedural syntax with powerful data description constructs based on associative arrays and extensible semantics. Lua is dynamically typed, runs by interpreting bytecode with a register-based virtual machine, and has automatic memory management with incremental garbage collection, making it ideal for configuration, scripting, and rapid prototyping.

— Lua supports an almost conventional set of statements, similar to those in C. This set includes assignments, control structures, function calls, and variable declarations.

— Control structures **if**, **else**, **while**, and **repeat** have the usual meaning and familiar syntax

— The for statement has two forms: one numeric and one generic. The numeric **for** loop repeats a block of code while a control variable runs through an arithmetic progression. Also **do** and **break** loops.

— Expressions: numbers and literal strings. Variables. Function definitions. Function calls. Table constructors. Both function calls and vararg expressions can result in multiple values.

— Arithmetic operators: the binary **+** (addition), **-** (subtraction), ***** (multiplication), **/** (division), **%** (modulo), and **^** (exponentiation); and unary **-** (negation). If the operands are numbers, or strings that can be converted to numbers, then all operations have the usual meaning.

— The relational operators are, **==** (equal), **~=** (not equal), **<** (less than), **>** (greater than), **<=** (less than or equal), **>=** (greater than or equal). These operators always result in false or true.

— Math operators like **Sin**, **Cos**, **Absolute**, **Log**, **Power**, and more.

— I/O and OS level file open close read write support and control.

Fuses use Lua for the Fuse language and further language documentation can be found on the Lua site and the LuaJit site.

Types of Fuse Plugins

Fuse plugins can be Image Processing and Metadata Processing or Modifiers or View LUT Plugins.

Image Processing Fuse plugins look like regular tools in Resolve and Fusion and can use all the same UI controls in the Inspector tool controls, and onscreen crosshairs and widgets are also available for use. Metadata processing is part of this Fuse type.

Modifier Plugins affect Number inputs and show up in the Modifier list. These are used instead of animation splines to control numbers of a slider or the center of a merge as an example.

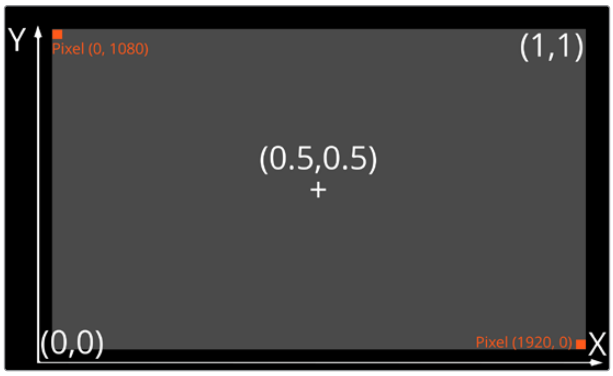
View Lut plugins are used to modify the image before being displayed, like linear to rec709 type color conversion or for utility like zebra striping to show out of range data. These are explained in the View Lut Plugin Reference.

Image Basics

Fuses do image processing and use internal functions in the Fusion engine to access images and pixels. UI tools and Onscreen controls are built in and can be animated just like normal tools.

Images

Images are 2D arrays of pixels with 2 axis, X and Y. Coordinates are normalized, bottom left of the image is (0,0) the center of the image is (0.5,0.5) and top right is (1,1). Images can also be accessed as pixels as well.



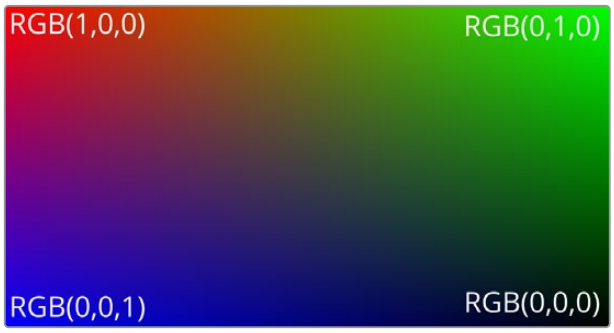
Aspect Ratio is supported for nonsquare pixels.

Proxy sizing is also a part of the process with sizes of the original image and proxy size available to the plugin.

PIxels are accessed directly in images in the X and Y directions from 0 to Width -1 for the X axis and 0 to Height -1 axis for the Y axis, the Bottom Left is pixel (0, 0) and Top Right is (Width-1. Height -1)

Color

Color is in RGB space, Alpha matte channel is always part of the image. There are also extended deep pixel channels like Z-Buffer, Vectors, Disparity and more as well. Color of pixels are in floating point numbers from 0 (black) to 1 (white), these values can be higher than 1 like in HDR and can also be negative.



Channels

Images can have AOV channels beyond Red Green Blue, the channels available are:

R, G, B, A	Red, Green, Blue and Alpha channels
BgR, BgG, BgB, BgA	Background Red, Green and Blue channels
Z	Z Buffer Channel
Coverage	Z buffer coverage channel
ObjectID, MaterialID	The ObjectID and MaterialID channels
U, V, W	U,V and W texture map coordinates channels
NX, NY, NZ	XYZ normal channels
VectorX, VectorY	The forward X and Y motion vector channels to the next frame
BackVectorX, BackVectorY	Back X and Y motion vectors to the previous frame
DisparityX, DisparityY	Per pixel Disparity position between 2 images, normally stereo images
PositionX, PositionY, PositionZ	World position channels

Canvas Color

Fusion engine supports infinite canvas for images, which is that image extends in all directions, the region beyond the bounds of the image data is set using the canvas color.

Image Domain

The Domain of Definition, abbreviated to DoD, refers to a rectangular region that defines what part of the image actually contains image data, this area can be outside of the viewing area and can be used to limit the area of processing to the relevant area to speed performance.

Metadata

Image metadata is supported and can be processed by Fuse plugins. It is a set of variables that are attached to an image and are passed alongside pixel data through the comp. Cameras store metadata inside the files like time code location, color data and lens data.

Fuse Plugin Programing

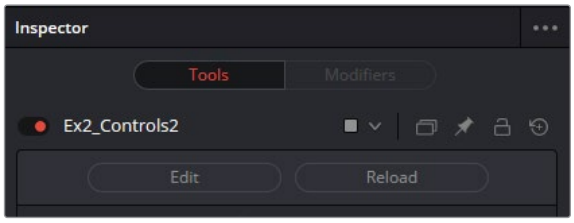
Fuse Plugins are generally composed of 3 classes

- FuRegisterClass function, to register and name your plugin, to show up in the tool list of the Fusion page and set the menu category. Brief description and link to help can also be set.
- UI controls are set in the Create Function. These are all animateable and are available in the Inspector tool control area and on screen.
- Process event is the heart of the plugin wear Image Processing code and algorithms are executed. There is a library of builtin image tools and functions to speed development.

Editing and Loading

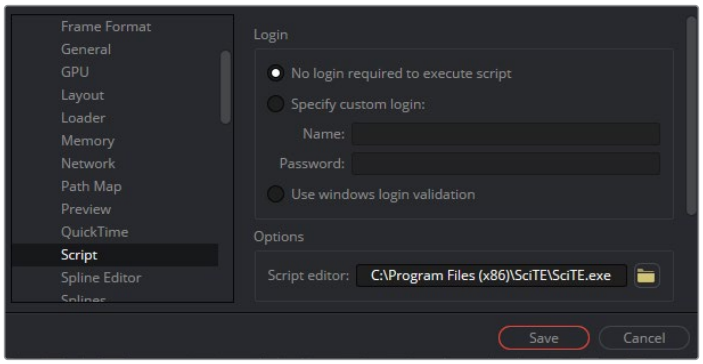
Fuse Plugins are compiled on the fly or just in time compiled. Reloading a Fuse will cause a load of the source code into cache and recompile.

Edit and Reload buttons are found at the top of the Inspector Tool Controls.



Setting Code Dev Editor

In Fusion Settings or Preferences the Code Editing Application Editing application and location can be set. This will allow any text editor to be associated to the Fuse files for editing and development.



Naming Conventions

There are 2 names, a label that is displayed in menus and Inspector Tool controls and internal names registering the plugin and tool controls internally, often the same name or similar abbreviation.

For example in creation register base class :

```
FuRegisterClass("ExampleColorCorrector", CT_Tool,
REGS_Name = "Ex1_BrightContrast",
```

The label displayed will be `"ExampleColorCorrector"` and internally this tool is referred to as `"Ex1_BrightContrast"`

In the UI create section control inputs are in the form `"Label"`, `"name"`.

```
InBright = self:AddInput("Brightness", "Bright"
```

In this example `"Brightness"` will be displayed on the Tool control and internally `"Bright"` will be used in the setup inside Fusion.

Note that a Variable number called InBright will be passed to the Process function of the plugin.

IMPORTANT The name should use only characters between A-Z, a-z, 0-9 and the underscore. Do Not use spaces in the naming and should not start with a number or use other special characters.

Common names

UI Tool Controls can have the same name and allow for compatibility of passing the same variables to each tool in a comp. If a Copy of a Tool node on a comp is `"Paste Setting"` onto another tool then common UI control values and animation will transfer to the same controls.

The common UI tool control names include Blend, Gain, Saturation, Brightness, Gamma for slider type controls and Angle, Center, Size, Pivot for image transform operations.

Variables

In C language variables are declared by *int*, *uint*, *float*, double etc, in Lua language this happens based on first use of the variable. To declare variables use the term local.

In this example local bright = `InBright:GetValue(req).Value` The declared variable is a number.

This is number local `r = 0` and could be Integer or floating point,Setting the variable to local `g = 0.0` will set it to float.

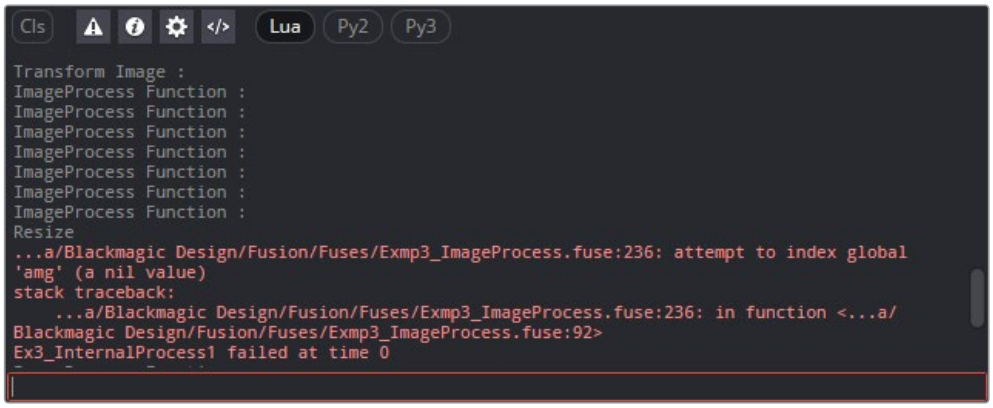
This is a pointer to an image `local img = InImage:GetValue(req)`

The following is a table of strings

```
local apply_operators = { "Over", "In", "Held Out", "Atop", "XOr", }
```

Console – Print and Debug

Resolve and Fusion have a Scripting Console which will show debug information and can also output text via the Print function.



The Print function can be formatted to combine strings and numbers and used for debug checking when code development is in process.

Other Programing Notes

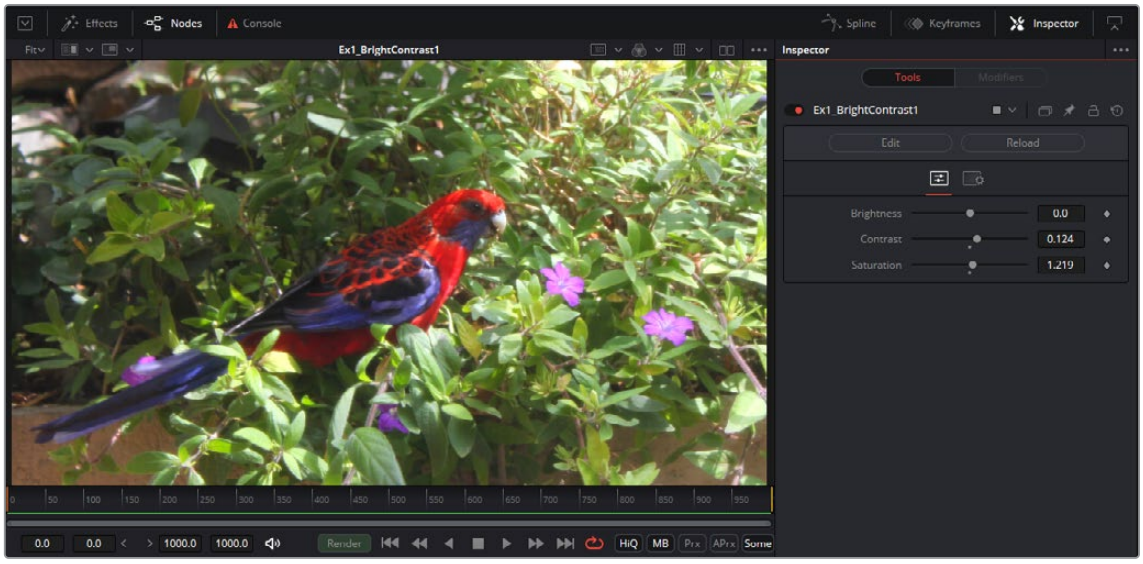
— \\ Double backslashes are used for Directories, to not confuse with other operations

— Comments are marked with a -- for a single line and --[[open bracket comment]]-- closed

Example 1 – Overview of a Fuse

This section will use **Example1_BrightContrast.fuse**, found in the Tools menu Fuses/Examples.

This outlines a basic Fuse and the 3 Function to create a tool node. The Register to name it, the Create for the UI controls, and Process for the image calculations.



This is a simple color corrector that has 3 slider controls for Brightness, Contrast and Saturation.

FuRegisterClass Function

The FuRegisterClass call describes the Fuse in a way that Fusion can recognize. This section tells Fusion the tool Name, location Tool menu, description and abbreviation.

This section should contain a single function call to FuRegisterClass(). The FuRegisterClass function requires three arguments. The first sets the tools name, the second sets the tools type, as defined in Fusion's internal registry, and the last argument is a table containing attributes which define the tools name, icon and other particulars.

This is a minimum to register a Fuse.

```
FuRegisterClass(ExampleColorCorrector", CT_Tool, {
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = E1BC",
    REGS_OpDescription = "Example1, using the functions of Color
Matrix",,
    })-- End of RegisterClass
```

More options and settings are available.

```
FuRegisterClass("ExampleColorCorrector", CT_Tool, {
    REGS_Name = "Ex1_BrightContrast",
    REGS_Category = "Fuses\\Examples",
    REGS_OpIconString = "E1BC",
    REGS_OpDescription = "Example1, using the functions of Color Matrix",
    REGS_HelpTopic = "Example Location of Help", --This can be a URL
    REGS_URL = "www.blackmagicdesign.com",
    REG_OpNoMask = true,
    REG_NoBlendCtrls = true,
    REG_NoObjMatCtrls = true,
    REG_NoMotionBlurCtrls = true,
    REG_NoBlendCtrls = false,
    REG_Fuse_NoEdit     = false,
    REG_Fuse_NoReload   = false,
    REG_Version = 1,
}) -- End of RegisterClass
```

UI Controls - Create Function

This is where parameter controls are defined, there are different types like sliders and onscreen crosshairs. The create function is run when the Fuse tool is added to the composition, or a composition containing that fuse tool is loaded. It describes the controls presented by the tools control window, and the inputs and outputs shown on the tools tile in the flow. The Create function takes no arguments.

It presents 3 sliders named 'Brightness', 'Contrast', 'Saturation' in the Inspector Tool control area , and the tool has one image input and one image output.

```
function Create()
    InBright = self:AddInput("Brightness","Brightness", { -- UI Label,
    Internal Ref
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_MaxScale = 1.0,
           INP_MinScale = -1.0,
           INP_Default = 0.0,
           })
    InContrast = self:AddInput("Contrast", "Contrast", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_MaxScale = 1.0,
           INP_MinScale = -1.0,
           INP_Default = 0.0,
           })

    InSaturation = self:AddInput("Saturation", "Saturation", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INP_MaxScale = 5.0,
           INP_MinScale = 0.0,
           INP_Default = 1.0,
           ICD_Center = 1,
           })

    InImage = self:AddInput("Input", "Input", {
           LINKID_DataType = "Image",
           LINK_Main = 1,
           })

    OutImage = self:AddOutput("Output", "Output", {
           LINKID_DataType = "Image",
           LINK_Main = 1,
           })

end -- end of Create()
```

Process Event Function

The Process Event function is where the image processing operations occur and is executed whenever Fusion asks the fuse tool to render a frame. Fusion's renderer will pass the Process function a single argument, an object called the Request. This argument contains all the information the tool needs to know about the current render environment. The image.at the current time and the Controls like sliders at the current time.

The following Process example, will get an image and assign it to a Variable 'img', it also gets three values from the sliders in the Inspector Tool controls and assigns them to variables 'bright', 'contrast',

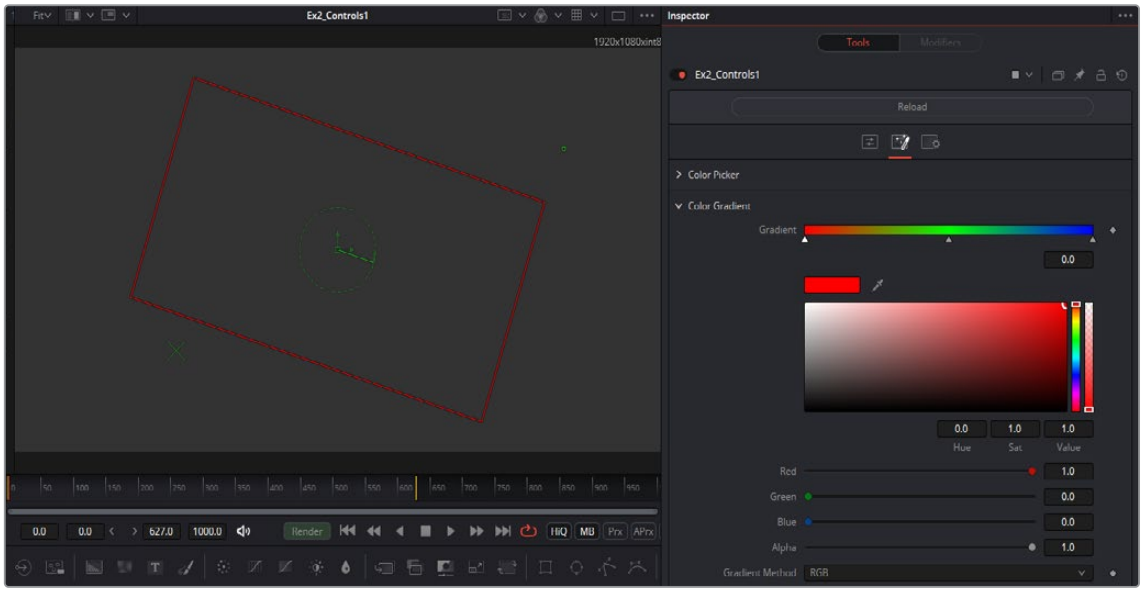
'sat'. It creates local variables 'r','g','b','a' and also creates a Color Matrix 'm'. Internal functions are used to manipulate the matrix and then runs the function ApplyMatrixOf to the image, outputting to a destination image, then finally sets the resulting image to the fuse tools output.

```
function Process(req)
    -- Get values from the UI Tools
    local img = InImage:GetValue(req)
    local bright = InBright:GetValue(req).Value
    local contrast = InContrast:GetValue(req).Value + 1
    local sat = InSaturation:GetValue(req).Value

    -- Define a set of variables
    local r = 0
    local g = 0
    local b = 0
    local a = 0

    if bright == 0 and sat == 1.0 and contrast == 1.0 then
           -- no change, go ahead and bypass this tool
           OutImage:Set(req, img)
    else
           -- create a color matrix
           local m = ColorMatrixFull()
           --Apply Brightness to the matrix via Offset function
           r = bright
           g = bright
           b = bright
           m:Offset(r, g, b, a)
           --Apply Contrast by offsetting the color to the midpoint 0.5
           r = contrast
           g = contrast
           b= contrast
           a = 1
           m:Offset(-0.5, -0.5, -0.5, -0.5)
           m:Scale(r, g, b, a)
           m:Offset(0.5, 0.5, 0.5, 0.5)
           --Apply Saturation by converting the Color Matrix to YUV and
           Scaling the Chroma UV channels
           m:RGBtoYUV()
           m:Scale(1, sat, sat, 1)
           m:YUVtoRGB()

           --Apply the Color Matrix to the image img to output image out
           out = img:ApplyMatrixOf(m, {})
           --Output the image
           OutImage:Set(req, out)
    end
end
```

Example 2 – UI Controls

This section will use Example2_UIControls.fuse, found in the Tools menu Fuses/Examples.

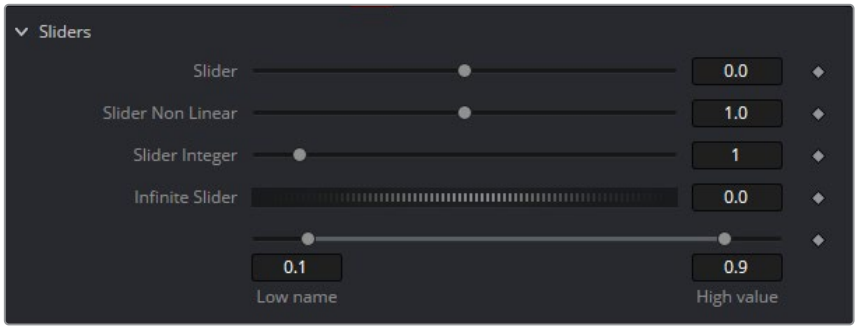
This Fuse presents the UI toolbox of controls and demonstrates features of the UI system. The code is commented for further information.



There are a rich assortment of UI controls available in the interface, showing in the Inspector Tool Control Area and on screen. This Fuse provides templates to all the controls as well as UI controls for Nesting control groups, Creating Tabs, and dynamically showing and hiding UI elements.

Sliders

Sliders are used to provide parameter inputs, and produce a number.



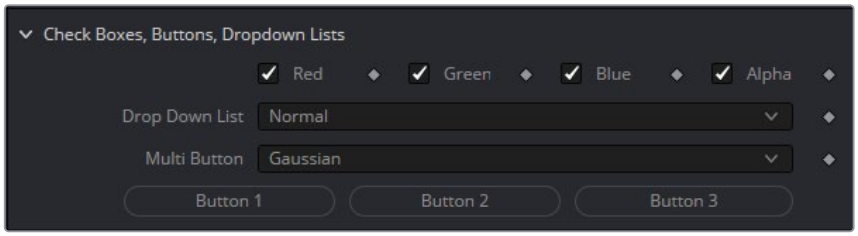
Sliders can be integer or floating point, can also be a fixed range or unlimited range, can be scaled or fixed scale, can have fixed limits and can also operate non linearly.

```
-- Slider Control returns a number
InSliderB = self:AddInput("Slider Non Linear", "SliderN", { -- UI Label,
Internal Ref
    LINKID_DataType = "Number", -- returns a number
    INPID_InputControl = "SliderControl", -- Type of Control
    INP_MaxScale = 5.0, -- Sets the default Maximum scale of the slider
```

```
    INP_MinScale = 0.0, --Sets the default Minimum  scale for the slider
    ICD_Center = 1,     -- Sets the default value to the center of the
    slider for Non Linear operation
    INP_Default = 1.0,  -- Sets default value for the slider
    INP_MinAllowed = 0, -- Sets the default Minimum value of the slider
    INP_MaxAllowed =10,-- Sets the default Maximum value of the slider
    })


-- Thumbwheel Screw Control is an Infinite slider, used for angle
controls, where fine control over values is needed and infinite numbers
can be set.
InScrewAngle = self:AddInput("Infinite Slider", "ScrewControl", {
LINKID_DataType = "Number",
INPID_InputControl = "ScrewControl",
    INP_MinScale = 0.0,
    INP_MaxScale = 100.0,
    INP_Default = 0,
    })
```

Thumbwheel Screw controls have infinite range yet still will give fine control, like for rotation type operations Range controls will result in High and Low values being returned.

Buttons, Check Boxes and Lists



Buttons are moment activation and do not change state, used for triggering events Check Boxes switch between states, returning 0 or 1.

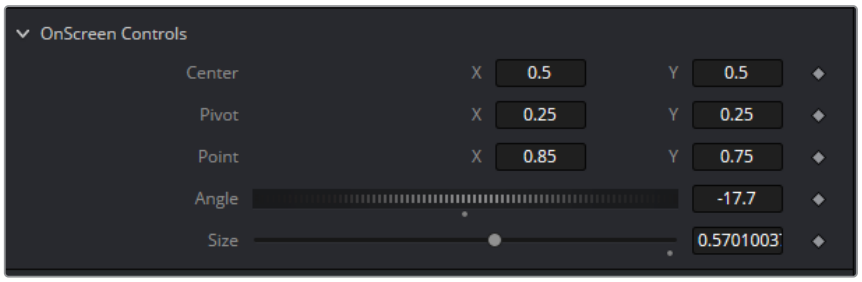
```
--Dropdown Lists are Combo Controls that will display and choose a
number of items. This is 17 items, and will return numbers 0 to 16
InDropList = self:AddInput("Drop Down List", "DropList", { --UI Label,
Internal Ref
    LINKID_DataType = "Number", -- returns a number
INPID_InputControl = "ComboControl", -- Type of Control
    INP_Default = 0.0,
    INP_Integer = true,
{ CCS_AddString = "Normal", }, -- labels for each option in the list
{ CCS_AddString = "Screen", },
{ CCS_AddString = "Dissolve", },
{ CCS_AddString = "Multiply", },
{ CCS_AddString = "Overlay", },
{ CCS_AddString = "Soft Light", },
```

```
{ CCS_AddString = "Hard Light", },
{ CCS_AddString = "Color Dodge", },
{ CCS_AddString = "Color Burn", },
{ CCS_AddString = "Darken", },
{ CCS_AddString = "Lighten", },
{ CCS_AddString = "Difference", },
{ CCS_AddString = "Exclusion", },
{ CCS_AddString = "Hue", },
{ CCS_AddString = "Saturation", },
{ CCS_AddString = "Color", },
{ CCS_AddString = "Luminosity",  },
    })
```

Drop Down Combo Control lists give unlimited selection of items from a list. Returns 0 for first, 1 for the second Multibutton is similar to Drop Lists, each option is displayed on a button to make the options visible.

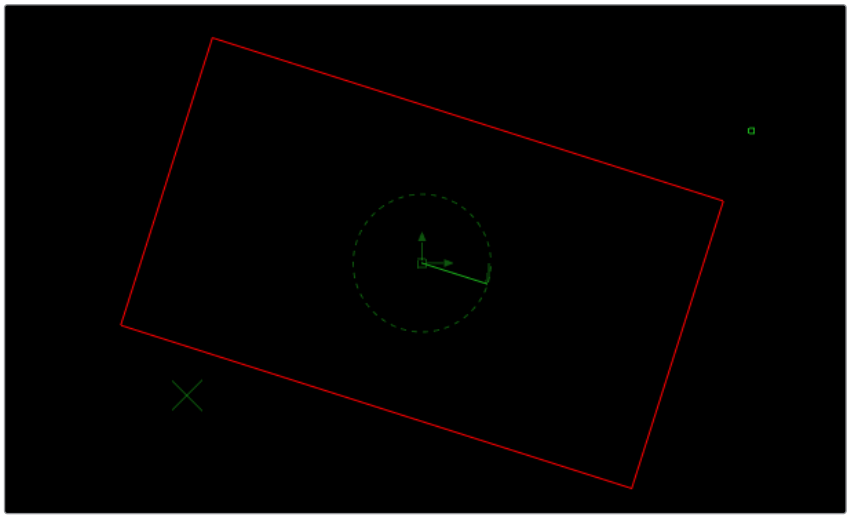
On Screen

These Point controls have 2 values for X and Y. On screen crosshairs, points and crosses can be defined, along with default position.




```
    -- Point controls are 2D used for on screen manipulation returning
    2 values X and Y
InCenter = self:AddInput("Center", "Center", { --UI Label, Internal Ref
LINKID_DataType      = "Point", -- Retrurns 2 values X and Y
INPID_InputControl   = "OffsetControl", -- Type of Control
INPID_PreviewControl = "CrosshairControl", -- Display Control
    INP_DefaultX          = 0.5,
    INP_DefaultY          = 0.5,
    })
```

Rectangle and Angle controls can be linked to Sliders and Thumbwheel Screw controls.

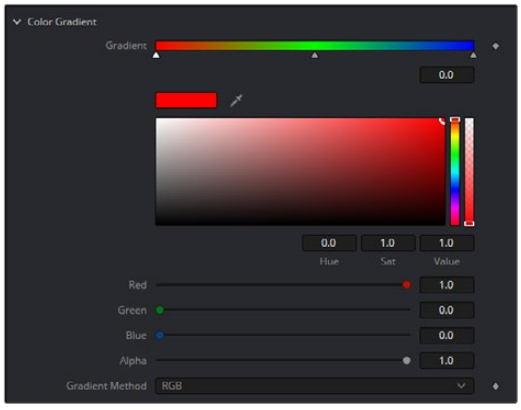


```
    -- The size Control is a slider with a connected onscreen Rectangle
    control
InSize = self:AddInput("Size", "Size", {
    LINKID_DataType = "Number",
    INPID_InputControl = "SliderControl",
            INP_Default = 1.0,
            })

    --Using Set Attributes, add OnScreen Rectangle connected to Center,
    Size & Angle
InSize:SetAttrs({
INPID_PreviewControl = "RectangleControl", -- Display Control Type
    RCP_Center = InCenter, -- Link to InCenter control above for location
    RCP_Angle = InAngle, -- Link Rotation to InAngle above
    RCD_LockAspect = 1.0,
    })
```

The onscreen widgets can be linked together, to have angle and rectangle controls associated with a Point Control. Selection priority can be set to make it easy to select overlapping controls.

Color Controls

The color Controls can be set up in a number of different ways.



Color control can show or hide the Color Wheel, and also set default color. Color picker is built into the color controls and these can pick RGBA or any auxiliary channel present.

```
    -- Color Control/Picker RGB sliders withe the Color Wheel/Swatch is
    hidden
InRed = self:AddInput("Red", "Red", { -- UI Label, Internal Ref
ICS_Name            = "Color",
LINKID_DataType     = "Number",
INPID_InputControl  = "ColorControl", -- Type of Control
    INP_Default          = 1.0,
    INP_MaxScale         = 1.0,
    CLRC_ShowWheel       = false,
    IC_ControlGroup      = 2, -- Groups Controls together. Make Group
    number it 2
    IC_ControlID         = 0,
    })
InGreen = self:AddInput("Green", "Green", { -- UI Label, Internal Ref
LINKID_DataType     = "Number",
INPID_InputControl  = "ColorControl", -- Type of Control
    INP_Default          = 0.9,
    IC_ControlGroup      = 2, -- Put this into a Group and number it 2
    IC_ControlID         = 1,
    })
InBlue = self:AddInput("Blue", "Blue", {--UI Label, Internal Ref
LINKID_DataType     = "Number",
INPID_InputControl  = "ColorControl", -- Type of Control
    INP_Default          = 0.6,
    IC_ControlGroup      = 2, -- Put this into a Group and number it 2
    IC_ControlID         = 2,
    })
```

Gradient Color

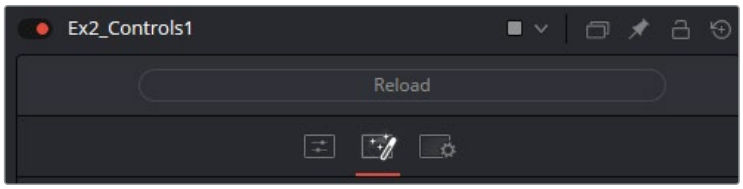
Gradient controls is a 1D array of color values that are used for color ramps. This has the color controls and picker built in.

```
    -- Gradient Color control has a 1D color ramp. Default Gradient is 2
    colors black to white. Use OnAddToFlow to set default colors
InGradient = self:AddInput("Gradient", "Gradient", { --UI Label, Internal Ref
LINKID_DataType   = "Gradient", -- Returns a Gradient 1D LUT
INPID_InputControl      = "GradientControl", -- Type of Control
    INP_DelayDefault     = true,
    })

InInterpolation = self:AddInput("Gradient Method", "GradientMethod", {
LINKID_DataType   = "FuID", -- Returns a FuID to the Color system
INPID_InputControl      = "MultiButtonIDControl", -- Type of Control
{ MBTNC_AddButton = "RGB", MBTNCID_AddID = "RGB", },
```

```
{ MBTNC_AddButton = "HLS", MBTNCID_AddID = "HLS", },
{ MBTNC_AddButton = "HSV", MBTNCID_AddID = "HSV", },
{ MBTNC_AddButton = "LAB", MBTNCID_AddID = "LAB", },
    MBTNC_StretchToFit   = true,
    INP_DoNotifyChanged = true,
INPID_DefaultID   = "RGB",
    })
```

Organize, Tabs, Nests

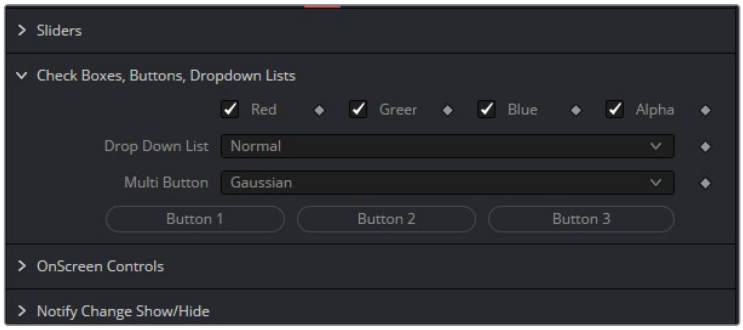
Tabs Appear across the top of the Inspector Tool Control area and allow the organisation of the UI Tool control inputs.



The creation of a Tab is a simple 1 line, and all controls following will be on this Tab page

```
    -- Control pages are new Tabs across the top of the tool controls
    in the Inspector tool control area
self:AddControlPage("Color Controls") -- Name the new Tab Control page
```

Nests allow the creation of groups of controls under a single heading. Nests can be twirled open or closed.



Nests are defined by a single line BeginControlNest and ended by the EndControlNest call

```
self:BeginControlNest("Color Picker", "ColorPicker", true); -- Control
Nests group controls with a togglable collapse/expand function

    -- More controls

    self:EndControlNest()
```

Image Inputs

A tool node can have a number of image inputs or outputs.

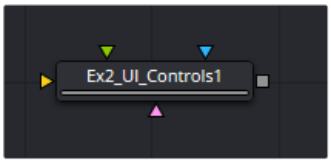
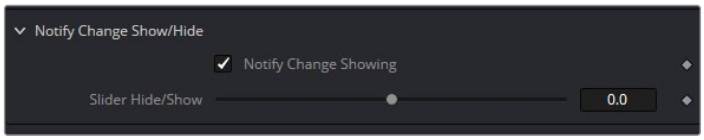


Image Inputs can be defined easily shown here in this code snippet.

```
--Image, Masks Inputs
    InImage = self:AddInput("Background", "Background", {
            LINKID_DataType = "Image",
            LINK_Main = 1,
            })
    InImage2 = self:AddInput("Image2", "Image2", {
            LINKID_DataType = "Image",
            LINK_Main = 2,
            INP_Required = false,
            })
 -- Create an output image
    OutImage = self:AddOutput("Output", "Output", {
            LINKID_DataType = "Image",
            LINK_Main = 1,
            })
```

Notify Change

The NotifyChanged event function is executed any time a control is changed on a tool. It executes before the Process event function. Typically the NotifyChanged event is used to adjust the values of controls before they are locked for rendering.



For example, the NotifyChanged function may be used to show and hide controls, or modify the name of a UI Label on a control in this example.

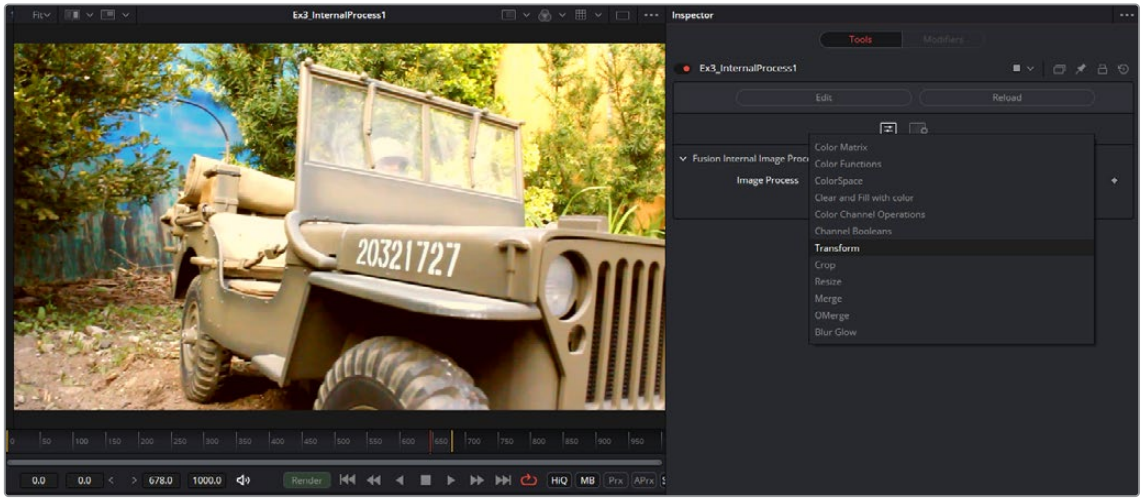
```
-- Notify Changed: Hide or Show controls and change Labels when options
   are selected
function NotifyChanged(inp, param, time)
-- If Notify Change check box changes then rename Control names and
   Un/Hide sliders
    if inp == InNotify then
            local locked = (param.Value > 0.5)
```

```
            if locked then
                  InSliderH:SetAttrs({ LINKS_Name = "Slider Hide/Show" })
                  InSliderH:SetAttrs({ IC_Visible = true })
                  InNotify:SetAttrs({ LINKS_Name = "Notify Change Showing" })
            else
                  InSliderH:SetAttrs({ LINKS_Name = "Slider Hide/Show Hidden" })
                  InSliderH:SetAttrs({ IC_Visible = false })
                  InNotify:SetAttrs({ LINKS_Name = "Notify Change Hidden" })
            end
      end
end
```

Example 3 – Internal Image Processing Functions

This section will use **Example3_ImageProcess.fuse and Exmp3_MultiPixelProcess.fuse.**

Ther Fusion engine has built in image processing functions, these are highly optimized and make it easy to do common operations like color, blur, merge and channel operations. Further info can be found in the `Process Reference.`



This has a dropdown list of Image processing functions with prefixed variables to show a result.

Each option in the Process Function is self contained and also for Channel operations will have multiple examples.

Color Matrix

Color Matrix uses a 4x4 matrix like 3D to manipulate color of an image, This can be used for linear operations like Brightness, Contrast, Gain and RGB-YUV conversion.

```
--********** Color  Matrix*********
local cm = ColorMatrixFull() -- Create a color matrix
--Apply Brightness to the matrix via Offset function
cm:Offset(0.1, 0.1, 0.1, 0) -- RGBA values adding 0.1 to RGB and 0 to Alpha
--Scale is the same as Gain which will multiply the Matrix by 1.1
cm:Scale(1.1, 1.1, 1.1, 1.1) -- Multiplies RGBA by 1.1 like Gain
--Applies the Color Matrix  (cm) on image (img) outputting to another
  image (out)
out = img:ApplyMatrixOf(cm, {})
```

Color Functions

Color Functions will apply basic color math operations to an image like Gain(multiply) , Gamma (power), Saturate (color +/- luminance), these can apply to RGBA extcepth Saturate.

```
--**********Color Functions***********
out = img:Copy() -- copy input image to out image
out:Gain(1.1 , 1.2 , 1.3, 1)--RGBA
out:Gamma(1.5, 1.5, 1.5, 1)--RGBA
out:Saturate(1.5,1.5,1.5)--RGB
```

Color Space

Color Space conversions can be applied to and from images, the RGB channels will store the converted image in the RGB channels, for example, HLS will be assigned as R =H, G=L, B=S.

```
--**********Color Space Conversion**********
--Image:CSConvert(<from>, <to>), with <from> and <to> coming from "RGB",
  "HLS", "YUV", "YIQ", "CMY", "HSV", "XYZ", "LAB".
out:CSConvert ("RGB", "YUV") --RGB to YUV where Y is in R, U is in G, V
                               is in B
out:Gamma(1.0, 0.9, 1.1, 1) --Example: Apply some Gamma to UV channels
out:CSConvert ("YUV", "RGB") -- Convert YUV to RGB
```

Clear and Fill Image

Clearing and Filling images with a set color can be done with Clear and Fill functions.

```
--**********Clear and Fill with color**********
out:Clear() -- Clears image set all channels to zero
out:Fill(Pixel({R = 1.0, G = 0.8, B = 0.25, A = 1.0})) -- Fill image with
                                                          color
```

Channel Operations

Channel operations are the arithmetic operations Add, Subtract, Multiply, and Divide. Values are applied to channels in the images.

```
--***********Color Channel Math Operations**********
-- Channel Math operations, apply a value to image(img) to output (out)
-- "Add", Multiply, Subtract, Divide
out = img:ChannelOpOf("Add", nil, { R = 0.1, G = 0.1, B = 0.1, A = 0.0})
out = out:ChannelOpOf("Multiply", nil, { R = 1.1, G = 1.1, B = 1.1, A =
1.0})
out = out:ChannelOpOf("Subtract", nil, { R = 0.2, G = 0.2, B = 0.2, A =
0.0})
out = out:ChannelOpOf("Divide", nil, { R = 0.8, G = 0.8, B = 0.8, A =
1.0})
```

Channel Booleans

Channel operations can also do more with 2 images, Foreground, Background and apply pixel for pixel operations Copy, Add, Multiply, Subtract, Divide, Threshold, And, Or, Xor, Negative, Difference, Signed Add.

```
-- Channel Boolean Math operations, pixel to pixel operations apply a
value to image(img) to output (out)
-- Copy, Add, Multiply, Subtract, Divide, Threshold, And, Or, Xor,
Negative, Difference, Signed Add


-- Multiply each pixel with a pixel from another image
out = img:ChannelOpOf("Multiply", fg, {R = "fg.R", G = "fg.G", B = "fg.B",
A = "fg.A"})
-- Add each pixel with a pixel from another image
out = img:ChannelOpOf("Add", fg, {R = "fg.G", G = "fg.R", B = "fg.B", A =
"fg.A"})
-- Threshold low-high
out = img:ChannelOpOf("Threshold", fg,{R="bg.R", G="bg.G", B="bg.B",
A="bg.A"}, 0.1, 0.8)
-- Copy fg channels to output
out = img:ChannelOpOf("Copy", fg, {R = "fg.R", G = "fg.G", B = "fg.B", A =
"fg.A"})
--Copy one channel to another, fg Red and bg Green and Blue
out = img:ChannelOpOf("Copy", fg, {R = "fg.R", G = "bg.G", B = "bg.B", A =
"fg.A"})
```

Transform

Transforming images, X&Y size, Angle, X&Y offset and pivot, as well as stamp or tiling rendering method.

```
--***********Transform Image**********
out = img:Transform(nil, {
```

```
    XF_XOffset = 0.65, --center.X,
    XF_YOffset = 0.6, --center.Y,
    XF_XAxis = 0.5, --pivot.X,
    XF_YAxis = 0.5, --pivot.Y,
    XF_XSize = 0.2, --sizex,
    XF_YSize = 0.4, --sizey,
    XF_Angle = 30.0, --angle,
    XF_EdgeMode = 1, --edge_modes  Black=0, Wrap(tile)=1, Duplicate edges=2
    })
```

Crop

Cropping or Cutting out a subsection of an image. Offsets are defined in pixels, and the destination image defines the size. Can also to the reverse and used to paste a smaller image into a larger image.

```
--**********Crop Image**********
--Original image (img) Output image (out). Offset in Pixels 0,0 is
bottom left. Negative and out of bounds values are allowed
img:Crop(out, {CROP_XOffset = 100, CROP_YOffset = 50}) -- Offset in pixels
```

Resize

Images can be resized to different sizes, defined by the X&Y size of the given output image, and different resize filter methods can be chosen.

```
--**********Resize Image**********
--Filter Methods:      Nearest, Box, Linear, Quadratic, Cubic, Catmull-
Rom, Gaussian, Mitchell, Lanczos, Sinc, Bessel
img:Resize(rszimg, {RSZ_Filter = "Cubic", })
```

Merge

Merge will overlay one image onto another, and has transform control over the foreground image as well as multiple apply modes for combining images and blending modes.

```
--**********Merge Foreground image over Background image**********
out:Merge(foreG, { --foreG image will be on top of out image
    MO_ApplyMode = apply_modes[applymode],
    MO_ApplyOperator = apply_operators[applyoperator],
    MO_XOffset = 0.75,  --center.X,
    MO_YOffset = 0.75,  --center.Y,
    MO_XAxis= 0.5,
    MO_YAxis =0.5,
    MO_XSize = 0.5 ,--xsize,
    MO_YSize = 0.5, --ysize,
    MO_Angle = 45, --angle,
    MO_FgAddSub = additive,
```

```
    MO_BgAddSub = additive,
    MO_BurnIn = 0.0, --burn,
    MO_FgRedGain   = 1.0,
    MO_FgGreenGain = 1.0,
    MO_FgBlueGain  = 1.0,
    MO_FgAlphaGain = 1.0,
    MO_Invert = 1,
    MO_DoZ = false,
    })
```

OMerge OXMerge

OMerge and OXMerge a Simple Additive or Subtractive Merge with Pixel XYOffsets. This can be used to reverse Crop Image.

```
-Merge will overlay the Foreground( img) over the copy Background image (out)
out:OMerge(foreG, 100, 50) -- Pixel integer offsets
out:OXMerge(foreG, 100, 50) -- Pixel integer offsets
```

Blur Glow

Blurring of images with different filter methods, color scale (gain) controls, blending and glow via the Normalize value.

```
--***********Blur Glow***********
img:Blur(out, { -- Blur will blur img into the result out
    BLUR_Type = 4,
    BLUR_Red = true ,
    BLUR_Green = true,
    BLUR_Blue =  true,
    BLUR_Alpha = true,
    BLUR_XSize = 10/720,
    BLUR_YSize = 10/720,
    BLUR_Blend = 0.5,
    BLUR_Normalize = 0.5,
    BLUR_Passes = 0.0,
    BLUR_RedScale = 1.0,
    BLUR_GreenScale = 1.0,
    BLUR_BlueScale = 1.0,
    BLUR_AlphaScale = 1.0,
    })
```

Example 4 – Multi Pixel Processing

Functions can be defined for pixel by pixel processing with 2 images and any image channel can be coded and multi threaded executed, see Example4_MultiPixelProcess.fuse.

All channel can be accessed, the names are : R, G, B, A, BgR, BgG, BgB, BgA, Z, Coverage, ObjectID, MaterialID, U, V, W, NX, NY, NZ, PositionX, PositionY, PositionZ, VectorX, VectorY, BackVectorX, BackVectorY, DisparityX, DisparityY.

Functions have to be defined before the Process in the code because there is no header files.

Creating Pixel Functions

Functions are defined to access 2 or more images on a pixel by pixel basis. p1 is the pixel of image 1, p2 is pixels from image 2. This Example uses a Function table to define 5 different functions and a simple drop down menu to select which function is processing.

```
--Function table for operations p1 is a pixel from image1 and p2 is a
pixel from image2
op_funcs =
{
    [1] =  function(x,y,p1,p2) -- min
                        p1.R = math.min(p1.R, p2.R)
                        p1.G = math.min(p1.G, p2.G)
                        p1.B = math.min(p1.B, p2.B)
                        p1.A = math.min(p1.A, p2.A)
                        return p1
                end,
    [2] =  function(x,y,p1,p2) -- max
                        p1.R = math.max(p1.R, p2.R)
                        p1.G = math.max(p1.G, p2.G)
                        p1.B = math.max(p1.B, p2.B)
                        p1.A = math.max(p1.A, p2.A)
                        return p1
                end,
    [3] =  function(x,y,p1,p2) -- add
                        p1.R = p1.R + p2.R
                        p1.G = p1.G + p2.G
                        p1.B = p1.B + p2.B
                        p1.A = p1.A + p2.A
                        return p1
                end,
    [4] =  function(x,y,p1,p2)
-- Variables. any number of variables can be named and passed to the
function
                        p1.R = gain * (p1.R - p2.R)
                        p1.G = p1.G - p2.G - bright
                        p1.B = var_C * (p1.B - p2.B)
                        p1.A = p1.A - p2.A
```

```
                        return p1
                end,
-- Copy Img2 RGB to Background and Normals Aux channels of img1
-- This is the all the channels available
-- R, G, B, A, BgR, BgG, BgB, BgA, Z, Coverage, ObjectID, MaterialID, U,
V, W, NX, NY, NZ
-- VectorX, VectorY, BackVectorX, BackVectorY, DisparityX, DisparityY,
PositionX, PositionY, PositionZ
    [5] =  function(x,y,p1,p2)
                        p1.R = p1.R
                        p1.G = p1.G
                        p1.B = p1.B
                        p1.A = p1.A
                        p1.BgR = p2.R
                        p1.BgG = p2.G
                        p1.BgB = p2.B
                        p1.BgA = p2.A
                        return p1
                end,
}
```

Process

This Process function shows expanded Image creation to define extra channels so there are 8 channels RGBA and Bg-RGBA.

MultiProcessPixels will use the defined function and apply it to the images.

NOTE That any number variables using any name can be passed to the functions, these do not have to be declared. In this case 3 variables gain, bright, var_C {gain = 2.0, bright=-0.3, var_C=1.5} are passed to the function.

```
function Process(req)
    local img1 = InImage1:GetValue(req)
    local img2 = InImage2:GetValue(req)
    local operation = InOperation:GetValue(req).Value+1
    if  img2 == nil then
          img2 = img1
    end
--This creates an image with 4 extra channels for function 5
     local imgattrs = {
          IMG_Document = self.Comp,
          { IMG_Channel = "Red", },
          { IMG_Channel = "Green", },
          { IMG_Channel = "Blue", },
          { IMG_Channel = "Alpha", },
```

```
            { IMG_Channel = "BgRed", },
            { IMG_Channel = "BgGreen", },
            { IMG_Channel = "BgBlue", },
            { IMG_Channel = "BgAlpha", },
            IMG_Width = img1.Width,
            IMG_Height = img1.Height,
            IMG_XScale = img1.XAspect,
            IMG_YScale = img1.YAspect,
            IMAT_OriginalWidth = img1.realwidth,
            IMAT_OriginalHeight = img1.realheight,
            IMG_Quality = not req:IsQuick(),
            IMG_MotionBlurQuality = not req:IsNoMotionBlur(),
            }
        if not req:IsStampOnly() then
            imgattrs.IMG_ProxyScale = 1
        end
        if SourceDepth ~= 0 then
            imgattrs.IMG_Depth = SourceDepth
        end
        local out = Image(imgattrs)

        local func = op_funcs[operation] -- get pointer to the function
        from the table

        -- Must have a valid operation function, and images must be same
        dimensions
    if func and (img1.Width == img2.Width) and (img1.Height == img2.Height)
       then out = Image({IMG_Like = img1})

       out:MultiProcessPixels(nil, {gain = 2.0, bright=-0.3, var_C=1.5}, 0,0,
       img1.Width, img1.Height, img1, img2, func)
    end
        OutImage:Set(req, out)
    end
```

Example 5 – Shapes, Lines, Text

This section refers to **Example5_Shapes.fuse** for defining and rendering of shapes, setting the color and styles. **Example4_Text.fuse** has info on text rendering.

It works similar to a 3D render context and there are 5 objects in creating and renderings a shape. A Shape is a linked list of lines or curves and defines as an outline or solid. FillStyle defines what style the shape will be filled with. ChannelStyle sets the color and blur, glow of the shape. Image channel defines an image or frame buffer to render shapes into it. Matrices are used to transform the shapes before rendering them into an image.

This sets up the context and creates a shape.

```
    local ic = ImageChannel(out, 8) -- Image Channel
    local fs = FillStyle() -- Fill Style Object
    local cs = ChannelStyle() -- Channel Style
    local mat = Matrix4() -- Matrix to transform the shapes
    local sh = Shape()


-- Shape made of a group of line segments
sh:MoveTo(0.078125, 0.1484375)
sh:LineTo(0.1748046875, -0.0078125)
sh:LineTo(0.177734375, -0.0732421875)
sh:LineTo(0.1455078125, -0.1435546875)
sh:LineTo(0.104777151878219, -0.160285078121503)
sh:LineTo(0.06640625, -0.150390625)
sh:LineTo(0.0068359375, -0.09375)
sh:LineTo(-0.05078125, -0.07421875)
sh:LineTo(-0.154296875, -0.1142578125)
sh:LineTo(-0.1767578125, -0.0986328125)
sh:LineTo(-0.1904296875, 0.0185546875)
sh:LineTo(-0.11328125, 0.0615234375)
sh:LineTo(-0.072265625, 0.1162109375)
sh:LineTo(-0.0546875, 0.1025390625)
sh:LineTo(-0.01953125, 0.1396484375)
sh:LineTo(0.0263671875, 0.1328125)
sh:LineTo(0.017578125, 0.1015625)
sh:LineTo(0.0625, 0.07421875)
```

The matrix is scaled, moved and rotated, the order of these operations is important, This example Scales first, moves the shape and rotates the entire shape.

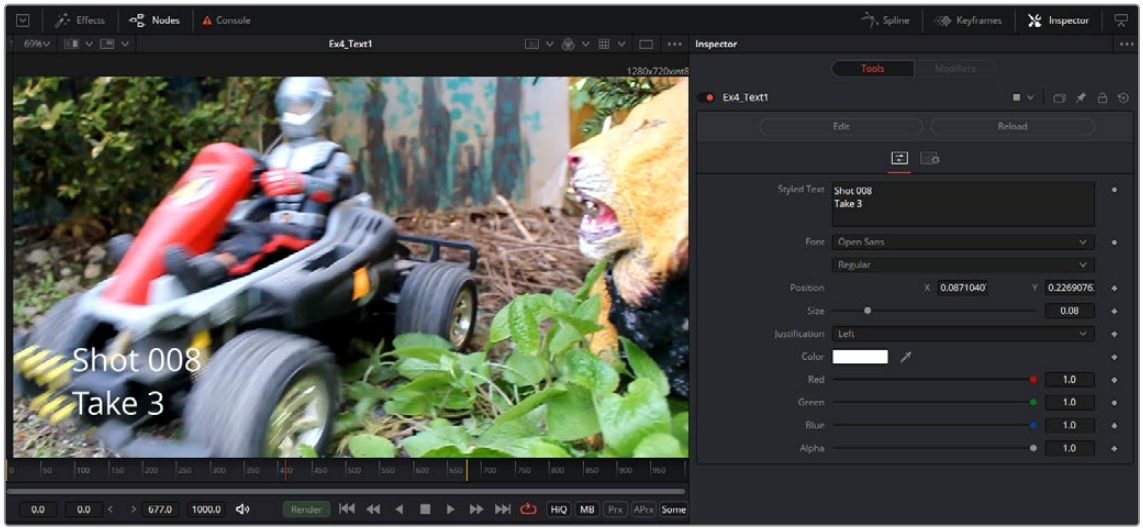
The second section sets the color and look, applies the matrix to the shape and renders it to an image using ShapeFill and PutToImage.

```
mat:Identity() -- Set the matrix to zero
mat:Scale(0.7, 0.7, 0.7) --Scale
mat:Move(0.25, 0.4, 0) -- Translate the Shape
mat:RotZ(-rotation) -- Rotate, Note the order of Matrix operations

cs.Color = Pixel{R=r , G=g , B=b, A = 1} -- Set the Color
ic:SetStyleFill(fs) -- Set the Drawing application styles
sh = sh:TransformOfShape(mat) -- Transform Shape using the Matrix
ic:ShapeFill(sh) --Apply Shape to the Image Channel
ic:PutToImage("CM_Merge", cs) --Render to the image
```

Example 6 – Text and Strings

This section will use **Example6_Text.fuse**. Text, Fonts, String handling and formatting, UI are all described in this Fuse that will give an text input UI widget and varour controls over the text.

System calls are used to query the OS for a fonts list to populate a list, and get font data.



UI Create

There are 3 UI input controls for text inputs, and will have a data type of text.

TextEditControl is a text input UI box that can be typed into as well as cut and paste.

This Fuse also shows creating a separate Function that can be called from Process.

```
function Create()
    InText = self:AddInput("Styled Text", "StyledText", {
            LINKID_DataType = "Text",
            INPID_InputControl = "TextEditControl",
            TEC_Lines = 3, -- How many lines high is the Input.
    })
    InFont = self:AddInput("Font", "Font", {
            LINKID_DataType = "Text",
            INPID_InputControl = "FontFileControl",
            IC_ControlGroup = 2,
            IC_ControlID = 0,
            INP_Level = 1,
            INP_DoNotifyChanged = true,
    })
    InFontStyle = self:AddInput("Style", "Style", {
            LINKID_DataType = "Text",
            INPID_InputControl = "FontFileControl",
            IC_ControlGroup = 2,
            IC_ControlID = 1,
            INP_Level = 1,
            INP_DoNotifyChanged = true,
    })
```

FontFileControl UI list controls will dynamically adjust list items and are used to get installed font lists from the OS. This can also be used to get the weight formatting of the font like Regular, Bold, and Light.

Process

The Process.Function in this Fuse will get 11 variables from the UI, Text strings, fonts and metrics, color and on screen location. If no font list is populated it will force the scanning of the Font list from the OS.

```
function Process(req)
    local img = InImage:GetValue(req)
    local font = InFont:GetValue(req).Value
    local style = InFontStyle:GetValue(req).Value
    local out = img:CopyOf()

    local text      = InText:GetValue(req).Value
    local size      = InSize:GetValue(req).Value
    local center    = InPosition:GetValue(req)
    local justify= InJustify:GetValue(req).Value
    local r         = InR:GetValue(req).Value
    local g         = InG:GetValue(req).Value
    local b         = InB:GetValue(req).Value
    local a         = InA:GetValue(req).Value

    local cx = center.X
    local cy = center.Y * (out.Height * out.YScale) / (out.Width * out.XScale)
    local quality = 32

-- if the FontManager list is empty, scan the font list
-- If the UI has never been shown, as would always be the case on a
render node,
-- nothing will scan the font list for available fonts. So we check for
that here,
-- and force a scan if needed.
    if not next( FontManager:GetFontList() ) then
          FontManager:ScanDir()
    end

    if req:IsQuick() then
          quality = 1
    end

    -- the drawstring function is doing all the heavy lifting
    drawstring(out, font, style, size, justify, quality, cx, cy,
    Pixel{R=r,G=g,B=b,A=a}, text)

    OutImage:Set(req, out)
end
```

Function Creation - Text Rendering

Fuses can have defined Functions that can be called from the other main Process function, These need to be defined before Process, so it knows to reference this function as there is no header file.

This code will Render Text to an image using the shape rendering functions built into the Fusion engine core. See the next section for further explanation.

```
   function drawstring(img, font, style, size, justify, quality, x, y,
   colour, text)
       local ic = ImageChannel(img, quality)
       local fs = FillStyle()
       local cs = ChannelStyle()

       cs.Color = colour
       ic:SetStyleFill(fs)

       -- get the fonts metrics
       local font = TextStyleFont(font, style)
       local tfm = TextStyleFontMetrics(font)

       -- This is the distance between this line and the next one.
local line_height=(tfm.TextAscent + tfm.TextDescent + tfm.
       TextExternalLeading) *10 * size
       local x_move = 0

       local mat = Matrix4()

       mat:Scale(1.0/tfm.Scale, 1.0/tfm.Scale, 1.0)
       mat:Scale(size, size, 1)

       -- set the initial baseline position of the text cursor
       local sh, ch, prevch

       local shape = Shape()
       mat:Move(x, y, 0)

       -- split the text into separate lines
       for line in string.gmatch(text, "%C+") do

             -- First pass, work out what the total width of this line is
             going to be
             local line_width = 0
             for i=1,#line do
                   ch = line:sub(i,i):byte()

                   -- is this ignoring kerning?
                   line_width = line_width + tfm:CharacterWidth(ch)*10*size
             end
```

```
-- Now work out our initial cursor position, based on the
justification
-- 0 = left justify,
-- 1 = centered
-- 2 = right justify
if justify == 0 then
	--mat:Move(0, 0, 0)
elseif justify == 1 then
	mat:Move(-line_width/2, 0, 0)
elseif justify == 2 then
	mat:Move(-line_width, 0, 0)
end

-- Second pass, now we assemble the actual shape
for i=1,#line do
	prevch = ch

	-- get the character, or glyph
	ch = line:sub(i,i):byte()

	-- first we want to know what the width of the character is,
	-- so we know where to start drawing this next character
	-- not really sure why we multiply this by 10, we just do :-)
	local cw = tfm:CharacterWidth(ch)*10*size

	-- if there is a previous character, we need to get
	the kerning
	-- between the current character and the last one.
	if prevch then
		x_offset = tfm:CharacterKerning(prevch, ch)*10*size
		x_move = x_move + x_offset
		mat:Move(x_offset, 0, 0)
	end

	-- move the cursor to the center of the character
	mat:Move(cw/2, 0, 0)

	-- I think this renders the shape we are interested in
	sh = tfm:GetCharacterShape(ch, false)
	sh = sh:TransformOfShape(mat)

	-- move the text cursor to the end of the glyph.
	mat:Move(cw/2, 0, 0)
	x_move = x_move + cw

	shape:AddShape(sh)
end
```

```
            -- line end, move the cursor back to the start
            if justify == 0 then
                  mat:Move(-x_move, -line_height, 0)
            elseif justify == 1 then
                  mat:Move(-x_move/2, -line_height, 0)
            elseif justify == 2 then
                  mat:Move(0, -line_height, 0)
            end

            x_move = 0
      end

      ic:ShapeFill(shape)
      ic:PutToImage("CM_Merge", cs)

end
```

Example 7 – Sampling

This section will use Example7_Sampling.fuse. Getting and Setting pixels and filtered sampling at any location is outlined in this example showing spatial warping of images and non linear transforms of pixels.

NOTE Images iterate for output image, so that every pixel can be filled.

Process Scatter

The Scatter function iterates through each output pixel and gets a source pixel based on the value red and blue channels shifting the position. The color channels return a float value between 0 and 1 and the Amplitude will control the strength of the effect. The output image is called avg.

```
print ("Scatter")
for y=0,img.Height-1 do
    for x=0,img.Width-1 do
            img:GetPixel(x,y, sp)
            xt = x - Amp * 5 * ( sp.R - 0.5)
            yt = y - Amp * 5 * ( sp.B - 0.5)
            if xt < 0 then
                  xt =0 end
            if xt > img.Width-1 then
                  xt =img.Width-1 end
            if yt < 0 then
                  yt =0 end
```

```
            if yt > img.Height-1 then
                  yt =img.Height-1 end

            img:GetPixel(xt,yt, sp)
            dp.R = sp.R
            dp.G = sp.G
            dp.B = sp.B
            dp.A = sp.A
            avg:SetPixel(x,y, dp)
    end
end
```

Process Sample

The Sample function iterates through each output pixel and gets a source pixel based on the Sine math function shifting the position. The output image is called avg

```
print ("Sample")
for y=0,img.Height-1 do
    for x=0,img.Width-1 do
            xt = x - ( Amp * math.sin((y * XF) + OffS))
            yt = y - ( Amp * math.sin((x * YF) + OffS))
            if xt < 0 then xt =0 end
            if xt > img.Width-1 then xt =img.Width-1 end
            if yt < 0 then yt =0 end
            if yt > img.Height-1 then yt =img.Height-1 end
            img:SamplePixelB(xt,yt, sp)
            dp.R = sp.R
            dp.G = sp.G
            dp.B = sp.B
            dp.A = sp.A
            avg:SetPixel(x,y, dp)
    end
end
```